

Greedy Graph Coverage for Bus Network Traversal

Ryan Pek

April 29, 2026

1 Problem Definition

We model the Singapore bus network as a directed graph:

- Vertices: bus stops
- Directed edges: valid next-stop transitions from bus routes

The goal is to construct a traversal that:

1. Visits every valid bus stop at least once
2. Minimizes total travel cost (initially edge count, later time)

This corresponds to a **graph coverage problem**:

Find a walk that visits all vertices at least once, minimizing repetition.

2 Baseline Approach

The simplest approach is greedy:

- From current stop, move to the nearest unvisited neighbour

This fails due to:

- Directed graph constraints
- Dead-ends and cycles
- Local minima (getting trapped in visited regions)

3 Improved Method: Path-Greedy Traversal

We define the following algorithm:

Algorithm

At each step:

1. Let v be the current stop
2. Find the nearest unvisited valid stop u using BFS (or Dijkstra)
3. Compute shortest path $P(v, u)$
4. Traverse P , allowing revisits
5. Mark all newly visited stops
6. Repeat until all valid stops are visited

If no path exists:

- Restart at another unvisited stop (new segment)

Key Idea

Replace local greedy decisions with global shortest-path jumps.

4 Graph Model

Let:

- $G = (V, E)$ be a directed graph
- V = set of bus stops
- E = directed edges from bus routes

Define valid stops:

$$V_{valid} = \{v \in V \mid \deg^+(v) > 0\}$$

5 Metrics

We evaluate algorithms using:

Coverage Ratio

$$\text{coverage} = \frac{\text{unique visited stops}}{|V_{valid}|}$$

Efficiency Ratio

$$\text{efficiency} = \frac{\text{tour length}}{\text{unique visited stops}}$$

Revisit Overhead

$$\text{overhead} = \frac{\text{tour length} - \text{unique visited stops}}{\text{unique visited stops}}$$

6 Experimental Result

For the Singapore bus graph:

- Valid stops: 5017
- Tour length: 8334

$$\text{efficiency} = \frac{8334}{5017} \approx 1.66$$

$$\text{overhead} \approx 66\%$$

This indicates:

- Full coverage achieved
- Moderate repetition due to graph structure constraints

7 Discussion

The main limitations arise from:

- Directed graph structure
- Lack of strong connectivity
- Local cycles

The algorithm avoids local traps by:

- Using shortest paths to escape visited regions
- Allowing revisits strategically

8 Future Work

Weighted Graph (Time Optimization)

Replace BFS with Dijkstra:

$$\text{edge weight} = \text{travel time}$$

Possible components:

- Bus travel duration
- Waiting time
- Transfer penalties

Graph Decomposition

Partition graph into clusters:

- Strongly Connected Components (SCCs)
- Weakly Connected Components
- Community clusters

Strategy:

1. Fully cover local cluster
2. Move between clusters via high-connectivity nodes

Centrality-Based Optimization

Use:

- Betweenness centrality to identify hubs
- Degree heuristics to guide traversal

Closed Tour Extension

To return to start:

- Append shortest path from final node back to start

This becomes:

Directed graph traveling salesman variant (asymmetric TSP)

9 Conclusion

We developed a practical graph traversal method:

- Greedy local strategies are insufficient
- Global shortest-path guidance is necessary
- Directed constraints introduce unavoidable overhead

The resulting algorithm:

Achieves full coverage with manageable repetition, and serves as a baseline for further optimization toward real-world travel efficiency.